

Reihenfolge bei der Sichtung der Daten

Pfade werden in `inline Code` angegeben. Die Pfade sind relativ zum Stammverzeichnis zu verstehen.

`/docs -> Dokumentation`

Innerhalb des Ordners `/docs` befindet sich die Dokumentation der Anforderungen.

Alle Bilder im Order `/docs/requirements-src/F**` sind ...

- Exporte aus der `*.plantuml`-Dateien, welche in `/docs/requirements-src/F**` Ordnern gefunden werden können
- Screenshots welche im Verlauf von Cypress E2E-Tests erzeugt wurden. (Die Tests sind vom Stammverzeichnis aus in `/apps/party-time-frontend-17/-e2e/src/e2e/` zu finden)

Sowohl die PlantUML Quellen als auch die Cypress Tests wurden im Rahmen des Praxisprojekts II & III von uns erstellt.

Anschauen sollten Sie sich in folgender Reihenfolge:

1. `/docs/_Anforderungen/anforderungen.pdf` -> umzusetzende Anforderungen
2. `/docs/other/design-entscheidungen.md` & `/docs/other/glossar.md` -> getroffene Designentscheidungen und Begründungen (kursive Begriffe können im Glossar nachgeschlagen werden)
3. `/docs/other/er-modell.plantuml` -> ER-Modell der Datenbank
4. jeden Ordner und den Inhalt `/docs/requirements-src/F**` -> Quell- Anforderungen und Diagramme zu den Anforderungen
5. jeden Ordner `/docs/.requirements/F**` -> Generierte Anforderungen und Diagramme zu den Anforderungen

In den Ordnern `/docs/requirements-src/F**` werden Bilder, Anforderungen als JSON und plantuml Diagramme gesammelt, welche mit den Tools im Ordner `/tools` zu den `.requirements` generiert wurden. Das Sichten dieser Ordner wäre redundant.

`/apps -> Anwendungen`

`/apps/party-time-frontend-17`

In dem Ordner befinden sich Unit-Test, E2E-Test und die Implementierung des Frontends.

1. `/apps/party-time-frontend-17/src/app` -> Implementierung des Frontends
2. `/apps/party-time-frontend-17/src/app/models` -> Interfaces und Typen

3. `/apps/party-time-frontend-17/src/app/services` -> Services & Tests für Service für die Kommunikation mit dem Backend (die Dokumentation inkl. Anforderungszugehörigkeit wird vom Interface vererbt und wird in der Implementierung nicht aufgeführt da redundant; die Test erreichen 100% Code Coverage; alle Pfade wurden getestet; spiegeln Swagger Dokumentation wieder)
4. `/apps/party-time-frontend-17/src/app/components` -> Globale Komponenten (Komponenten welche von jedem verwendet werden können)
5. `/apps/party-time-frontend-17/src/app/pages` -> Seitenkomponenten (Komponenten welche von den Routern verwendet werden; werden nur bei Bedarf geladen, Abhängigkeiten haben keine weitere Anforderungszugehörigkeit da sich diese transitiv ergibt)
6. `/apps/party-time-frontend-17/src/app/validators` -> Validatoren für Formulare (Testet die Passwörter auf Einhaltung der Regeln)
7. `/apps/party-time-frontend-17/tailwind.config.ts` -> Konfiguration für TailwindCSS

Anmerkungen zum Frontend

- es wurde auf eine Lazy Loading Strategie gesetzt, um die Ladezeiten zu minimieren
- die Anwendung ist vollständig responsive
- die Anwendung wurde mit TailwindCSS gestaltet
- die Anwendung wurde mit Angular 17 erstellt
- eine Seite kann lokale Komponenten verwenden, welche von anderen Seiten nicht verwendet werden

`/apps/party-time-backend-kotlin`

In dem Ordner befinden sich die Implementierung & Tests des Backends.

Das Spring-basierte Backend folgt einem Model-View-Controller Design.

Implementierung:

1. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime`
-> Implementierung des Backends
2. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/api/dto`
-> Data-Transfer-Objects, welche das Backend für Serialisierung und Deserialisierung von Daten verwendet, welche mit dem Frontend ausgetauscht werden
3. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/api/controller`
-> Controller des Backends, enthalten API Routen welche dem Frontend zur Verfügung gestellt werden
4. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/api/error`
-> DTO und Exceptions für Fehler, sowie Funktionalität welche Kotlin-Exceptions in Fehler-DTO umwandeln kann

5. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/service`
-> Services des Backends, werden von Controllern verwendet um die Anfragen zu bearbeiten
6. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/jpa`
-> Datenbank-Definitionen des Backends, werden von Services verwendet um Datenbankzugriffe zu tätigen
7. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/configuration`
-> OpenAPI Konfiguration, sowie diverse Sicherheits-Konfigurationen (Json-Web-Token, Spring-Security, etc.)
8. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/mail`
-> Zentrales Mail-handling des Backendes, implementiert mit Spring ApplicationEvent
9. `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/util`
-> Daten-Generator, welcher in Anwendungen die nicht mit Spring "prod" (Produktion) Profil laufen, Daten der Datenbank hinzufügt. Dies erleichtert das Testen und Entwickeln des Backends und Frontends, da bei einem Start der Anwendung nicht erst manuell Datensätze angelegt werden müssen
10. `/apps/party-time-backend-kotlin/src/main/resources` -> Enthält Properties-Dateien (in der Form von YAML-Dateien), welche das Backend nutzt um Konfigurationen zu laden. Die Haupt-Datei `application.yml` wird dabei von zusätzlichen `.yml` Dateien in bestimmten Einsatz-Szenarien unterstützt. Beispielsweise ermöglicht es die `application-mem.yml` das Backend mit einer In-Memory Datenbank zu starten.
11. `/apps/party-time-backend-kotlin/src/main/resources/mail` -> Enthält Mustache Templates für die E-Mails, welche das Backend versenden kann

Tests:

1. `/apps/party-time-backend-kotlin/src/test/kotlin/com/partytime`
-> Tests des Backends
2. `/apps/party-time-backend-kotlin/src/test/kotlin/com/partytime/service`
-> Tests der Services (100% Code Coverage)
3. `/apps/party-time-backend-kotlin/src/test/kotlin/com/partytime/scenario`
-> Tests die mithilfe von Szenarien im Stil von Customer-Journeys die Funktionalität des Backends testen. Ähneln integration-tests stark, aber haben mehr die Absicht die Interaktion von verschiedenen Nutzern (Beispiel: ein Nutzer erstellt ein Event, lädt einen anderen Nutzer ein, der andere Nutzer akzeptiert die Einladung, etc.) zu testen. (für 100% Code Coverage der Controller verantwortlich)
4. `/apps/party-time-backend-kotlin/src/test/kotlin/com/partytime/testAbstraction`
-> Enthält abstrakte Klassen die bereits mit benötigten Annotationen versehen wurden. Werden von den tatsächlichen Tests verwendet um die Test-Struktur zu vereinheitlichen.

Anmerkungen zum Backend

- Der E-Mail Versand funktioniert in der Theorie. Im Praxis-Projekt II funktionierte der E-Mail Versand mit vergleichbarem Code, der sich jetzt auch im Backend befindet, allerdings ist Herr Dangel nicht mehr Teil des Projektteams und er war derjenige, dem sich um den API Key des Backends kümmerte. Das Backend gibt “versendete” E-Mail Inhalte aktuell in nicht “prod” Profil-starts auf der Konsole aus. Aus diesem Grund ist die Code-Coverage in `/apps/party-time-backend-kotlin/src/main/kotlin/com/partytime/mail` nicht 100%. Wir bitten um Verständnis für diesen Umstand und uns ist natürlich bewusst, das für ein Produktivsystem der E-Mail Versand funktionieren muss und getestet werden muss.
- Die Code-Coverage mittels Tests ist im Backend sehr hoch. Sie ist nicht 100% da nicht alle Inhalte getestet werden können. Manche Code-Abschnitte entsprechen einfach Best-Practices (Bsp.: `isEquals` und `hashCode` von Klassen überschreiben, ohne das wir jemals von diesen Funktionen Gebrauch machen) und werden so gut wie möglich von dem Coverage Report ausgeschlossen. Vom Report ausgeschlossen werden Inhalte, die mit `@ExcludeFromCoverage` annotiert sind, sowie Inhalte, die in `/apps/party-time-backend-kotlin/build.gradle.kts` in Zeile 102 - 108 konfiguriert sind.
 - `DatabaseConstants` und `APIConstants` sind Konstanten, die nicht getestet werden können, da sie nur Konstanten enthalten
 - `PartyTimeBackendKotlinApplicationKt` enthält den Spring-Boot Startpunkt und wird von Spring-Boot beim Applikations-Start aufgerufen. Spring-Boot Tests führen diesen Code nicht aus. Es wurde sich bei dem Code so stark an die Vorgaben des Frameworks gehalten, dass er eigentlich unter die “Nicht die Framework-Funktionalität-testen” Regel von Ihnen fällt.
 - `AuthEntryPointJwtKt` ist **nicht** das gleiche wie `AuthEntryPointJwt`. `AuthEntryPointJwt` wird getestet, wohingegen die mit `Kt` endende Klasse nur eine von Kotlin erstellte Klasse ist, die `AuthEntryPointJwt`-externe Inhalte enthält. In diesem Fall ist dies nur ein Logger, dessen Initialisierung in Tests nicht getestet werden kann.

Testabdeckung

Die generierten Reports können Sie jeweils in dem Ordner `/coverage/apps/party-time-frontend-17` und `/coverage/apps/party-time-frontend-17/party-time-backend-kotlin` einsehen. Dazu öffnen Sie jeweils die Datei `index.html` im Web-Browser.

Sonstiges

1. `/package.json` -> listet die NPM Pakete auf, die das Projekt benötigt
2. `/nx.json` -> Konfiguration für den NX Workspace
3. `/tsconfig.base.json` -> Konfiguration für den Typescript Compiler

Anmerkungen

Folgende Anmerkungen sind zu beachten:

- F007 (Einladung erhalten), F017 (Benachrichtigung zum Event), F019 (Kalendereintrag exportieren) haben keinen Frontend-Teil und können nicht E2E getestet werden
- Folgende Daten, welche sich keinen Anforderungen direkt zuordnen lassen, haben weder Anforderungsnummer noch Kommentar:
 - generierte Dateien
 - config Dateien
 - Basisdaten für die Anwendungen
 - Testdaten für die Anwendungen
 - Daten für die Dokumentation
 - Model Klassen
 - Interfaces
 - Typen
- Code-nahe Dokumentation / Datenbankmodell ist in englisch gehalten
- Diagramme welche sich auf die Anforderungen beziehen sind in deutsch gehalten

Präsentation

Die Präsentation wurde ergänzt und kann zusätzlich zur Dokumentation eingesehen werden (Reihenfolge egal). Die Präsentation enthält eine Zusammenfassung der Anforderungen und deren Tests inklusive der Ablauf- und Sequenzdiagramme sowie Screenshots der Anwendung.